

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>

#define MaxData 100
#define MaxRow 10
#define MaxCol 20
int OriginalData[MaxData + 1];
int Data[MaxData + 1];
int Radix[MaxRow][MaxCol];
int N = 12;
/* =====
   Common Function
   =====
*/
void PrepareRawData(int N)
{
    int i, j;
    int Number;
    int Duplicate;
    for (i = 1; i <= N; i++)
    {
        do
        {
            Number = 1 + rand() % 99;
            Duplicate = 0;
            for (j = 1; j < i; j++)
            {
                if (OriginalData[j] == Number)
                {
                    Duplicate = 1;
                    break;
                }
            }
        } while (Duplicate == 1);
        OriginalData[i] = Number;
    }
}

void CopyOriginalData(int N)
{
    int i;

```

```

        for (i = 1; i <= N; i++)
            Data[i] = OriginalData[i];
    }
void DispOriginalData(int N)
{
    int i;
    for (i = 1; i <= N; i++)
        printf("%2d ", OriginalData[i]);
    printf("\n");
}
void DispData(int N)
{
    int i;
    for (i = 1; i <= N; i++)
        printf("%2d ", Data[i]);
    printf("\n");
}
/* =====
1. BUBBLE SORT
ใช้โค้ดเดิม
=====
*/
void BubbleSort(int N)
{
    int i, j, temp;
    printf("\n===== BUBBLE SORT
=====\\n");
    printf(" i ");
    for(i = 1; i <= N; i++)
        printf(" (%2d)", i);
    printf("\n");
    printf("-----\\n");
    printf("%2d. ", 0);
    DispData(N);
    for(i = 1; i < N; i++)
    {
        if(Data[i] > Data[i + 1])
        {
            j = i + 1;
            while(j > 1 && Data[j] < Data[j - 1])
            {

```

```

        temp = Data[j];
        Data[j] = Data[j - 1];
        Data[j - 1] = temp;
        printf("%2d. ", i);
        DispData(N);
        j = j - 1;
    }
}
}

/* =====
2. INSERTION SORT
ใช้โค้ดเดิม
=====
*/
void InsertionSort(int N)
{
    int i, j, temp;
    printf("\n===== INSERTION SORT
=====\\n");
    printf(" i ");
    for(i = 1; i <= N; i++)
        printf(" (%2d)", i);
    printf("\\n");
    printf("-----\\n");
    printf("%2d. ", 0);
    DispData(N);
    for (i = 1; i < N; i++)
    {
        if (Data[i + 1] < Data[i])
        {
            temp = Data[i + 1];
            Data[i + 1] = 0;
            j = i;
            while (j >= 1 && temp < Data[j])
            {
                Data[j + 1] = Data[j];
                Data[j] = 0;
                printf("%2d. ", i + 1);
                DispData(N);
                j = j - 1;
            }
        }
    }
}

```

```

        }
        Data[j + 1] = temp;
        printf("%2d. ", i + 1);
        DispData(N);
    }
}

/* =====
3. HEAP SORT
ใช้โครงสร้างเดิม
=====
*/

int Maximum(int a, int b)
{
    if (Data[a] > Data[b])
        return a;
    else
        return b;
}

void AdjustTree(int LastNode)
{
    int i, MaxChild;
    i = 1;
    while (2 * i <= LastNode)
    {
        if (2 * i + 1 <= LastNode)
            MaxChild = Maximum(2 * i, 2 * i + 1);
        else
            MaxChild = 2 * i;
        if (Data[i] < Data[MaxChild])
        {
            int temp;
            temp = Data[i];
            Data[i] = Data[MaxChild];
            Data[MaxChild] = temp;
            i = MaxChild;
        }
        else
            break;
    }
}

void CreateHeapTree(int LastNode)

```

```

{
    int i;
    for (i = LastNode / 2; i >= 1; i--)
        AdjustTree(LastNode);
}

void HeapSort(int N)
{
    int i;
    int temp;
    printf("\n===== HEAP SORT =====\n");
    CreateHeapTree(N);
    printf("Heap Tree : ");
    DispData(N);
    for (i = N; i >= 2; i--)
    {
        temp = Data[1];
        Data[1] = Data[i];
        Data[i] = temp;
        printf("Swap : ");
        DispData(N);
        AdjustTree(i - 1);
    }
    printf("Sorted Data : ");
    DispData(N);
}

/* =====
4. QUICK SORT
ใช้โครงสร้างเดิม
=====
*/

void swap(int a, int b)
{
    int temp;
    temp = Data[a];
    Data[a] = Data[b];
    Data[b] = temp;
}

void QuickSort(int f, int r)
{
    int f1, r1;
    bool direction;
    f1 = f;

```

```

    r1 = r;
    direction = true;
    while (f != r)
    {
        if (Data[f] > Data[r])
        {
            printf("%2d %2d : ", f, r);
            DispData(N);
            swap(f, r);
            printf("%2d %2d : ", f, r);
            DispData(N);
            direction = !direction;
        }
        if (direction)
            r--;
        else
            f++;
    }
    if ((f > f1) && (f - 1 != f1))
        QuickSort(f1, f - 1);
    if ((r < r1) && (r + 1 != r1))
        QuickSort(r + 1, r1);
}

/* =====
5. RADIX SORT
ใช้โครงสร้างเดิม
=====
*/

void ClearStackPT()
{
    int i;
    for (i = 0; i < MaxRow; i++)
        Radix[i][0] = 0;
}

void Push(int Rad, int Dat)
{
    int Location;
    Location = Radix[Rad][0] + 1;
    Radix[Rad][Location] = Dat;
    Radix[Rad][0] = Location;
}

void ReadStack(int N)

```

```

{
    int i, j, k;
    k = 1;
    for (i = 0; i < MaxRow; i++)
    {
        for (j = 1; j <= Radix[i][0]; j++)
        {
            Data[k] = Radix[i][j];
            k++;
        }
    }
}

void RadixSort(int N)
{
    int Digit;
    int i;
    int RadixNo;
    int Divisor;
    printf("\n===== RADIX SORT =====\n");
    Divisor = 1;
    for (Digit = 1; Digit <= 2; Digit++)
    {
        ClearStackPT();
        for (i = 1; i <= N; i++)
        {
            RadixNo = (Data[i] / Divisor) % 10;
            Push(RadixNo, Data[i]);
        }
        ReadStack(N);
        printf("Digit %d : ", Digit);
        DispData(N);
        Divisor = Divisor * 10;
    }
    printf("Sorted Data : ");
    DispData(N);
}

/* =====
6. SELECTION SORT
ใช้โครงสร้างเดิม
=====
*/

int Minimum(int j, int N)

```

```

{
    int i;
    int Location;
    Location = j;
    for (i = j + 1; i <= N; i++)
    {
        if (Data[i] < Data[Location])
            Location = i;
    }
    return Location;
}

void SelectionSort(int N)
{
    int i;
    int Location;
    int temp;
    printf("\n===== SELECTION SORT\n");
    printf(" i ");
    for(i = 1; i <= N; i++)
        printf(" (%2d)", i);
    printf("\n");
    printf("-----\n");
    printf("%2d. ", 0);
    DispData(N);
    for (i = 1; i < N; i++)
    {
        Location = Minimum(i, N);
        if (Location != i)
        {
            temp = Data[i];
            Data[i] = Data[Location];
            Data[Location] = temp;
        }
        printf("%2d. ", i);
        DispData(N);
    }
    printf("Sorted Data : ");
    DispData(N);
}

/* =====

```

```

MAIN PROGRAM
=====
*/
int main()
{
    int choice;
    srand(time(NULL));
    /* สุ่มข้อมูลครั้งแรก */
    PrepareRawData(N);
    CopyOriginalData(N);

    printf("=====
    =====\n");
    printf("                SORTING ALGORITHMS PROGRAM\n");

    printf("=====
    =====\n");
    printf("Initial Random Data : ");
    DispOriginalData(N);
    do{ printf("\n=====
    =====\n");
        printf("                SORTING ALGORITHMS\n");
    printf("=====
    =====\n");
        printf("Current Random Data : ");
        DispOriginalData(N);
        printf("\n");
        printf("1. Bubble Sort\n");
        printf("2. Insertion Sort\n");
        printf("3. Heap Sort\n");
        printf("4. Quick Sort\n");
        printf("5. Radix Sort\n");
        printf("6. Selection Sort\n");
        printf("7. Generate New Random Data\n");
        printf("0. Exit\n");
        printf("\nChoose : ");
        scanf("%d", &choice);
        switch(choice)
        {
            case 1:
                CopyOriginalData(N);
                BubbleSort(N);

```

```

        break;
    case 2:
        CopyOriginalData(N);
        InsertionSort(N);
        break;
    case 3:
        CopyOriginalData(N);
        HeapSort(N);
        break;
    case 4:
        CopyOriginalData(N);
        printf("\n===== QUICK SORT
=====\\n");
        QuickSort(1, N);
        printf("Sorted Data : ");
        DispData(N);
        break;
    case 5:
        CopyOriginalData(N);
        RadixSort(N);
        break;
    case 6:
        CopyOriginalData(N);
        SelectionSort(N);
        break;
    case 7:
        PrepareRawData(N);
        CopyOriginalData(N);
        printf("\\nNew Random Data : ");
        DispOriginalData(N);
        break;
    case 0:
        printf("\\nExit Program...\\n");
        break;
    default:
        printf("\\nPlease choose 0 - 7 only.\\n");
    }
} while(choice != 0);
return 0;
}

```

กระบวนการทำงานของโปรแกรมในโหมด Selection Sort

เริ่มต้นฟังก์ชันจะนำข้อมูลที่สุ่มชุดตัวเลขที่มีการเก็บไว้ใน Array[] จาก ฟังก์ชัน PrepareRawData(N) และนำข้อมูลที่สุ่มเข้าตัวแปร Data[i] สำหรับเตรียมเข้าฟังก์ชันในการเลือกจำนวนต่อไป

```
void SelectionSort(int N)
{
    int i;
    int Location;
    int temp;

    printf("\n===== SELECTION SORT\n");
    printf(" i ");
    for(i = 1; i <= N; i++)
        printf(" (%2d)", i);
    printf("\n");
    printf("-----\n");
    printf("%2d. ", 0);
    DispData(N);
    for (i = 1; i < N; i++)
    {
        Location = Minimum(i, N);
        if (Location != i)
        {
            temp = Data[i];
            Data[i] = Data[Location];
            Data[Location] = temp;
        }
        printf("%2d. ", i);
        DispData(N);
    }
    printf("Sorted Data : ");
    DispData(N);
}
```

ในฟังก์ชัน void SelectionSort(int N) จะรับข้อมูล N เพื่อนำไปเข้าสู่กระบวนการทำงานตามจำนวนข้อมูลที่มีการสุ่มเตรียมไว้ในที่แรก แสดงชุดข้อมูลที่สุ่มมาในที่แรกออกมาให้หมดเพื่อให้เห็นว่าข้อมูลปัจจุบันยังไม่ได้ถูกจัดเรียงอย่างถูกต้องและบอกกำกับลำดับตำแหน่งข้อมูลในตัว จากนั้นเข้าสู่ลูปถัดไปเพื่อเตรียมจัดเรียงลำดับชุดข้อมูล โดยกระบวนการทำงานของ SelectionSort

คือการทำงานเรียงที่ละตำแหน่งแล้วไล่หาข้อมูลที่มีน้อยกว่าข้อมูลปัจจุบันแล้วนำมาแทนที่ตำแหน่งนั้น

```
int Minimum(int j, int N)
{
    int i;
    int Location;

    Location = j;
    for (i = j + 1; i <= N; i++)
    {
        if (Data[i] < Data[Location])
            Location = i;
    }
    return Location;
}
```

กระบวนการดังกล่าวนี้อยู่ในฟังก์ชัน `int Minimum(int j, int N)` ซึ่งรับข้อมูล 2 ข้อมูลคือ ลำดับที่ปัจจุบัน กับจำนวนข้อมูลทั้งหมด เริ่มลูบการทำงานครั้งแรกหมายถึงตำแหน่งที่ 1 และวนไปข้อมูลถัดไปจนสุดท้าย แล้วเก็บข้อมูลที่มีเงื่อนไขดังนี้ `Data[i] < Data[Location]` หมายถึงตำแหน่งการทำงานที่เก็บข้อมูลมีค่าข้อมูล น้อยกว่า ตำแหน่งตัวเทียบหรือไม่ หากเป็นจริงจะนำข้อมูลนั้นมาแทนที่แล้ววนลูบไปเรื่อยๆจนพ้นเงื่อนไข `i <= N` จนนั้นค่าข้อมูลที่ได้จะเป็นค่าที่ถูกนำมาแทนที่ตำแหน่งที่ 1 แล้วสลับข้อมูลตำแหน่งไปยัง ณ ข้อมูลที่เจอ เมื่อกระบวนการทำงานครบจะได้ชุดข้อมูลที่เรียงจาก น้อย ไป มาก ในที่สุด มีลักษณะผลลัพธ์ดังนี้

```

=====
                        SORTING ALGORITHMS PROGRAM
=====
Initial Random Data : 27 50 12 11  8 67 85 21 46 23 32 52

=====
                        SORTING ALGORITHMS
=====
Current Random Data : 27 50 12 11  8 67 85 21 46 23 32 52

1. Bubble Sort
2. Insertion Sort
3. Heap Sort
4. Quick Sort
5. Radix Sort
6. Selection Sort
7. Generate New Random Data
0. Exit

Choose : 6

===== SELECTION SORT =====
i  ( 1) ( 2) ( 3) ( 4) ( 5) ( 6) ( 7) ( 8) ( 9) (10) (11) (12)
-----
0. 27 50 12 11  8 67 85 21 46 23 32 52
1.  8 50 12 11 27 67 85 21 46 23 32 52
2.  8 11 12 50 27 67 85 21 46 23 32 52
3.  8 11 12 50 27 67 85 21 46 23 32 52
4.  8 11 12 21 27 67 85 50 46 23 32 52
5.  8 11 12 21 23 67 85 50 46 27 32 52
6.  8 11 12 21 23 27 85 50 46 67 32 52
7.  8 11 12 21 23 27 32 50 46 67 85 52
8.  8 11 12 21 23 27 32 46 50 67 85 52
9.  8 11 12 21 23 27 32 46 50 67 85 52
10. 8 11 12 21 23 27 32 46 50 52 85 67
11. 8 11 12 21 23 27 32 46 50 52 67 85
Sorted Data :  8 11 12 21 23 27 32 46 50 52 67 85

```

จากนั้นโปรแกรมก็จะวนกลับมาที่หน้าเมนูเหมือนเดิมและรอรับคำสั่งเลือกประเภทจัดเรียงข้อมูลจากผู้ใช้ต่อไป

โดยกระบวนการทำงานเรียงข้อมูลแบบ Selection

มีหลักการที่เข้าใจง่ายและสามารถเข้าโปรแกรมได้ง่าย แต่ก็มีเวลากระบวนการทำงานที่ช้าเป็น

$O(n^2)$ ไม่ว่าจะในกรณีที่ข้อมูลมากน้อยขนาดไหนก็จะใช้เวลากระบวนการเท่าเดิม